

Converting MyBible Translation Modules for Mobil-Bible (KaiOS)

This guide explains how to use `bible_converter.py` to turn a standard **MyBible** (**.SQLite3**) translation module into the JSON format that the **Mobil-Bible** KaiOS app reads — either as the bundled default translation shipped inside the app package, or as an additional translation loaded from the phone's microSD card.

This version of the guide uses a **fixed, simple folder structure** so that users who are not comfortable with computers can follow it step by step without getting lost in paths.

1. What the converter does

`bible_converter.py` reads a MyBible `.SQLite3` file (which contains a `books` table and a `verses` table) and produces:

- `info.json` — a small index listing every book's ID, short name, long name, and chapter count. This is loaded once into memory for navigation.
- **One `[book_id].json` file per book** — the actual scripture text, split by chapter and verse. These are loaded one at a time on demand, so the phone never has to hold the whole Bible in RAM at once.

All HTML/XML markup found in MyBible text (`<b>`, `<i>`, `<pb/>`, etc.) is stripped automatically, and extra whitespace is cleaned up.

2. System requirements

The converter is a single Python 3 script with **no external dependencies** — it only uses Python's built-in `sqlite3`, `json`, `os`, `re`, and `argparse` modules. You do **not** need to install anything beyond Python itself.

Linux Mint

Python 3 is pre-installed on Linux Mint. Just confirm it's available:

```
❏  
bash  
python3 --version
```

If it prints a version number (e.g. `Python 3.10.x` or newer), you're ready. If for some reason it's missing:

```
❏  
bash  
sudo apt update  
sudo apt install python3
```

Windows 10 / Windows 11

Python is **not** installed by default on Windows, so you need to install it once:

1. Download the latest Python 3 installer from <https://www.python.org/downloads/windows/>.
2. Run the installer. On the first screen, **make sure to tick "Add python.exe to PATH"** before clicking "Install Now" — this step is easy to miss and is required so you can run `python` from the command line.
3. After installation, open **Command Prompt** (or **PowerShell**) and verify:

```
❏  
powershell  
python --version
```

(On Windows, the command is usually `python`, not `python3`.)

No other libraries, compilers, or packages are needed — `sqlite3` ships with Python's standard library on all three platforms.

3. Recommended folder structure (do this once)

To keep things simple and avoid typing long, error-prone paths, set up two fixed folders **before** you convert anything. You only need to do this once — after that, every conversion follows the exact same pattern.

Windows 10 / 11

Create two folders directly on your `C:` drive:

Folder

Purpose

`C:\MyBible` Put the converter script (`bible_converter.py`) **and** all your `.SQLite3` translation files here

`C:\Bibles` Leave this empty — the converter will automatically create a subfolder here for every translation you convert

You can create both folders in File Explorer (right-click on the `C:` drive → **New** → **Folder**), or from PowerShell:

```
❏  
powershell  
mkdir C:\MyBible  
mkdir C:\Bibles
```

Then:

1. Copy `bible_converter.py` into `C:\MyBible`.
2. Copy every `.SQLite3` file you want to convert (e.g. `ELBBK.SQLite3`, `NIVUK.SQLite3`) into `C:\MyBible` as well.

From now on, **everything happens inside** `C:\MyBible`, and every result automatically lands inside `C:\Bibles`.

Linux Mint

Use the same idea, but based in your **home folder** (this is your personal "root" — the true system root `/` requires administrator rights and should not be used for this):

Folder

Purpose

`~/MyBible` (i.e. `/home/<your-username>/MyBible`)

Put the converter script (`bible_converter.py`) **and** all your `.SQLite3` translation files here

`~/Bibles` (i.e. `/home/<your-username>/Bibles`)

Leave this empty — the converter will automatically create a subfolder here for every translation you convert

Create both folders from a terminal:

```
❏  
bash  
mkdir -p ~/MyBible  
mkdir -p ~/Bibles
```

Then copy `bible_converter.py` and your `.SQLite3` files into `~/MyBible` (drag-and-drop in the file manager works fine too).

4. Getting a MyBible module

You need a MyBible-format translation file with the extension `.SQLite3` (e.g. `ESV.SQLite3`, `KJV.SQLite3`, `Karoli.SQLite3`). These modules follow the standard MyBible schema, with a `books` table (book numbers, short/long names) and a `verses` table (book, chapter, verse, text).

Once you have downloaded a module, put it straight into your `MyBible` folder from Section 3 (`C:\MyBible` on Windows, `~/MyBible` on Linux Mint) — nowhere else.

The converter auto-detects the correct column names even if a module uses slightly different column naming conventions (`id`/`book_number`, `name`/`long_name`, etc.), so most standard MyBible modules will work without modification.

5. Step-by-step conversion

5.1 Open a terminal in the right folder

Windows: Open **File Explorer**, navigate into `C:\MyBible`, then click into the address bar, type `cmd` (or `powershell`) and press **Enter**. This opens a terminal that is already "standing" inside `C:\MyBible`.

Linux Mint: Open a terminal, then type:

```
❏  
bash  
cd ~/MyBible
```

5.2 Run the conversion

Because your files now always live in the same two fixed folders, the command is always the same shape — only the file name and translation name change.

Windows (run from inside `C:\MyBible`):

```
❏  
powershell  
python bible_converter.py NIVUK.SQLite3 NIVUK --out C:\Bibles
```

Linux Mint (run from inside `~/MyBible`):

```
bash
python3 bible_converter.py NIVUK.SQLite3 NIVUK --out ~/Bibles
```

Explanation of the three parts you may need to change for each translation:

- `NIVUK.SQLite3` — the exact file name of the module sitting in your `MyBible` folder.
- `NIVUK` — the name you want the translation to be called in the app (this becomes the subfolder name, and the name shown in **Options → Select Translation**). Use short, simple names with no spaces or accented characters (e.g. `NIVUK`, `ELBBK`, `KJV`).
- `--out C:\Bibles` (Windows) / `--out ~/Bibles` (Linux) — tells the converter to put the result straight into your fixed `Bibles` folder. **Always include this**, so you never have to search for the output afterwards.

The converter automatically creates a subfolder for the translation inside `Bibles`, e.g. `C:\Bibles\NIVUK\`, containing `info.json` and one `.json` file per book.

Repeat this same command (with a different `.SQLite3` file and translation name) for every additional translation you want to convert — they will simply pile up as separate subfolders inside `Bibles`, e.g.:

```
C:\Bibles\
├── ELBBK\
├── NIVUK\
└── KJV\
```

5.3 (Advanced, optional) Converting only part of the Bible

If you specifically want to bundle only the New Testament inside the app package itself (the built-in "Default Bible"), see Section 7. For normal SD-card translations, you do **not** need any extra options beyond `--out` — just follow 5.2 above.

If you ever need to exclude the Old or New Testament from an SD-card translation, two optional switches exist:

Option	Purpose
<code>--min-book</code> <code>N</code>	Only include books with book number $\geq$ N (e.g. <code>470</code> = start of New Testament)
<code>--max-book</code> <code>N</code>	Only include books with book number $\leq$ N

Example — convert only the Old Testament:

```
❏  
powershell  
python bible_converter.py Karoli.SQLite3 Karoli --max-book 460 --out C:\Bibles
```

The script prints progress for each book as it converts, and a final summary once done.

6. Where to copy the converted files (onto the phone's SD card)

Thanks to the fixed `--out C:\Bibles` (or `--out ~/Bibles`) folder used in every command above, **every converted translation is already exactly where it needs to be** — you never have to go looking for it.

1. Insert the phone's microSD card into your computer (card reader, or the phone itself connected via USB in storage mode).
2. Copy the **entire** `Bibles` **folder** (from `C:\Bibles` on Windows, or `~/Bibles` on Linux Mint) to the **root of the microSD card**, so the final result on the card looks like:

```
❏  
  
/sdcard/Bibles/ELBBK/info.json  
/sdcard/Bibles/ELBBK/10.json  
/sdcard/Bibles/NIVUK/info.json  
/sdcard/Bibles/NIVUK/470.json  
..
```

or, depending on how KaiOS mounts the card:

```
❏  
  
/storage/sdcard/Bibles/...
```

3. Put the card back in the phone.
4. On the phone: **Options → Select Translation** — every translation you converted should now appear in the list, alongside the built-in Default Bible.

Reminder: for this to work on the device, `manifest.webapp` must declare the `device-storage:sdcard` permission and `"type": "privileged"`.

You can keep adding more translations at any time: just repeat Section 5.2 for a new `.SQLite3` file (still using `--out C:\Bibles` / `--out ~/Bibles`), then re-copy the whole `Bibles`

folder onto the card again — existing translations on the card will simply be overwritten with the same content, and the new one will be added alongside them.

7. Bundling a translation inside the app itself (advanced, optional)

Most users only need Section 5–6 (SD-card translations). This section is only relevant if you are the app developer and want to change the **built-in Default Bible** that ships inside the app package (loaded from `bible/default/` even without an SD card).

To regenerate it — typically with the New Testament only, to keep the app small:

Windows:

```
❏  
powershell  
python bible_converter.py NIVUK.SQLite3 default --min-book 470 --out C:\bible
```

Linux Mint:

```
❏  
bash  
python3 bible_converter.py NIVUK.SQLite3 default --min-book 470 --out ~/bible
```

This creates a `default` subfolder (note the different output root, `bible` instead of `Bibles`) containing `info.json` and `470.json`, `480.json`, etc. Copy this `default` folder into the Mobil-Bible project itself, replacing the existing one, so it sits alongside `index.html`, `style.css`, and `app.js`:

```
❏  
mobil-bible/  
├── index.html  
├── style.css  
├── app.js  
├── manifest.webapp  
└── bible/  
    ├── default/  
    │   ├── info.json  
    │   ├── 470.json  
    │   └── ...
```

This becomes part of the app package that gets installed via WebIDE's "Install and Run" (or packaged into the final `.zip`/build for the app store), so it will always be available offline even without an SD card.

8. Quick verification checklist

After converting and copying files, confirm:

1. `info.json` is valid JSON (a plain array of `{ "id", "short", "full", "chapters" }` objects) — you can open it in any text editor to sanity-check it.
2. Each book file is named exactly `<book_id>.json` (matching the `id` values in `info.json`), with no extra prefixes or extensions.
3. For SD-card translations: the folder sits directly under `Bibles/<TranslationName>/` at the card root (not nested deeper).
4. For the bundled translation: the folder is exactly `bible/default/` relative to `index.html`.
5. Launch the app (Simulator or physical device) and check that the new translation appears and opens without console errors.

9. Troubleshooting

Symptom	Likely cause
<code>[Errno 2] No such file or directory</code> when running the command	You are not standing inside the <code>MyBible</code> folder, or the <code>.SQLite3</code> file / <code>bible_converter.py</code> isn't actually there. Re-check Section 3 and re-open the terminal from inside <code>C:\MyBible</code> (or <code>~/MyBible</code>).
<code>Error: The database does not contain standard MyBible 'books' or 'verses' tables.</code>	The <code>.SQLite3</code> file isn't a standard MyBible module, or it's corrupted — verify it opens correctly in a MyBible-compatible reader first.
Converter finishes but a book is missing	That book had no verses in the source database (the script logs a <code>[Warning] No verses found for book N. Skipping.</code> line and continues) — this is expected if the source module is partial (e.g. NT-only).
Translation doesn't show up in the app's SD card list	Check the folder is directly under <code>Bibles/</code> at the card root, and that <code>info.json</code> exists inside the translation's own subfolder.

Symptom

Likely cause

Bundled translation didn't update after re-copying `bible/default/`

Make sure you replaced the whole folder (old per-book `.json` files from a previous build can remain if you only copied `info.json`) and reinstall/reload the app so cached files aren't reused.

PowerShell shows a `Get-Process` / `ParameterBindingException` error

The prompt text (e.g. `PS C:\Users\...>`) got pasted in front of the actual command by mistake. Only paste the part starting with `python`.